Name: Atharva Waghmode
Roll No. - 59
Div - D15A

Experiment 2 – To Perform Data Preprocessing Using Python

Aim

To perform data preprocessing on a synthetic Student Result dataset using Python by handling missing values, removing duplicate records, encoding categorical data, and scaling numerical features.

Objectives

- Load the student result dataset using Python.
- Identify and handle missing values.
- Remove duplicate records.
- Convert categorical data into numerical form.
- Normalize/standardize numerical data.
- Prepare clean data for further analysis or machine learning.

Software/Tools Required

- Python 3.x
- Jupyter Notebook / Google Colab
- Pandas
- NumPy
- Scikit-learn

Theory

Data preprocessing prepares raw data for analysis by correcting incomplete, inaccurate, or inconsistent values and arranging the information in a usable format for analytics and machine learning.

The main stages generally include:

- Data Cleaning – Handling missing and incorrect values.
- Removing Duplicates – Eliminating repeated records.
- Encoding – Converting categorical values into numerical values.
- Feature Scaling – Bringing numerical features to a similar scale.
- Data Transformation – Converting data into a suitable format.

Dataset

A synthetic dataset of 500 records was generated to represent student academic activity in the College Database System, mirroring the Result_Fact schema designed in Experiment 1. The dataset was intentionally seeded with missing values and duplicate records to give every preprocessing step meaningful data to act on.

Columns: student_id, age, class, section, study_hours_per_week, attendance_pct, subject, city, marks_obtained

Output

1. Loading the Dataset

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import LabelEncoder, StandardScaler

np.random.seed(42)
n = 500

classes = ['9th', '10th', '11th', '12th']
sections = ['A', 'B', 'C']
subjects = ['Maths', 'Science', 'English', 'Computer', 'History']
cities = ['Mumbai', 'Pune', 'Delhi', 'Bangalore', 'Chennai']

df = pd.DataFrame({
    'student_id': np.arange(1001, 1001 + n),
    'age': np.random.randint(13, 18, n).astype(float),
    'class': np.random.choice(classes, n),
    'section': np.random.choice(sections, n),
    'study_hours_per_week': np.round(np.random.uniform(2, 20, n), 1),
    'attendance_pct': np.round(np.random.uniform(50, 100, n), 1),
    'subject': np.random.choice(subjects, n),
    'city': np.random.choice(cities, n),
    'marks_obtained': np.round(np.random.uniform(35, 100, n), 1),
})

# Seed some missing values
for col in ['age', 'study_hours_per_week', 'attendance_pct', 'city']:
    idx = np.random.choice(df.index, size=np.random.randint(15, 35), replace=False)
    df.loc[idx, col] = np.nan

# Seed some duplicate rows
dupes = df.sample(12, random_state=1)
df = pd.concat([df, dupes], ignore_index=True)

df.to_csv('student_result_dataset.csv', index=False)

# Load the dataset
df = pd.read_csv('student_result_dataset.csv')
print(f"Dataset Shape: {df.shape}")
df.head()
```

```
Dataset Shape: (512, 9)
```

	student_id	age	class	section	study_hours_per_week	attendance_pct	subject	city	marks_obtained
0	1001	NaN	10th	B	17.1	95.1	English	Chennai	95.5
1	1002	17.0	12th	C	17.2	NaN	Science	Delhi	67.3
2	1003	15.0	12th	A	19.2	NaN	English	Pune	50.0
3	1004	17.0	9th	C	4.8	93.4	History	Bangalore	37.9
4	1005	17.0	10th	B	13.2	68.8	History	Delhi	62.0

Dataset loaded successfully – Shape: (512, 9). The head shows missing values (NaN) already present in Age and Attendance_pct.

2. Checking Duplicates and Missing Values

```python
print(f"Total Duplicate Rows: {df.duplicated().sum()}\n")
print("Missing Values Per Column:")
print(df.isnull().sum())
```

```
Total Duplicate Rows: 12

Missing Values Per Column:
student_id              0
age                    32
class                   0
section                 0
study_hours_per_week    26
attendance_pct          30
subject                 0
city                    27
marks_obtained          0
dtype: int64
```

12 duplicate rows found; missing values present in age (32), study_hours_per_week (26), attendance_pct (30), and city (27).

3. Removing Duplicate Records

```
df = df.drop_duplicates()
print(f"Shape after removing duplicates: {df.shape}")
```

```
Shape after removing duplicates: (500, 9)
```

Shape after removing duplicates: (500, 9).

4. Handling Missing Values

```
numeric_cols = ['age', 'study_hours_per_week', 'attendance_pct']
for col in numeric_cols:
    df[col] = df[col].fillna(df[col].mean())

df['city'] = df['city'].fillna(df['city'].mode()[0])

print("Missing values after imputation:")
print(df.isnull().sum())
```

```
Missing values after imputation:
student_id              0
age                     0
class                   0
section                 0
study_hours_per_week    0
attendance_pct          0
subject                 0
city                    0
marks_obtained          0
dtype: int64
```

Numerical columns (age, study_hours_per_week, attendance_pct) were imputed using the column mean, and the categorical column (city) was imputed using the mode. No missing values remain in any column.

5. Encoding Categorical Data

```
categorical_cols = ['class', 'section', 'subject', 'city']
label_encoder = LabelEncoder()

for col in categorical_cols:
    df[col] = label_encoder.fit_transform(df[col])

df[categorical_cols].head()
```

	class	section	subject	city
0	0	1	1	1
1	2	2	4	2
2	2	0	1	4
3	3	2	2	0
4	0	1	2	2

class, section, subject, and city were encoded into numeric labels using LabelEncoder.

6. Feature Scaling

```
scaler = StandardScaler()
scale_features = ['age', 'study_hours_per_week', 'attendance_pct', 'marks_obtained']

df[scale_features] = scaler.fit_transform(df[scale_features])
df[scale_features].describe().round(2)
```

	age	study_hours_per_week	attendance_pct	marks_obtained
count	500.00	500.00	500.00	500.00
mean	0.00	-0.00	-0.00	0.00
std	1.00	1.00	1.00	1.00
min	-1.42	-1.79	-1.83	-1.80
25%	-0.70	-0.80	-0.88	-0.84
50%	0.03	0.00	-0.00	0.00
75%	0.76	0.83	0.83	0.89
max	1.48	1.73	1.83	1.72

age, study_hours_per_week, attendance_pct, and marks_obtained were standardized to mean = 0, std = 1 across all 500 rows.

7. Final Preprocessed Dataset Info

```
print(f"Final Preprocessed Shape: {df.shape}\n")
print(df.info())
```

```
Final Preprocessed Shape: (500, 9)

<class 'pandas.core.frame.DataFrame'>
Index: 500 entries, 0 to 499
Data columns (total 9 columns):
 #   Column               Non-Null Count  Dtype  
---  ------               --------------  -----  
 0   student_id           500 non-null    int64  
 1   age                  500 non-null    float64
 2   class                500 non-null    int64  
 3   section              500 non-null    int64  
 4   study_hours_per_week 500 non-null    float64
 5   attendance_pct       500 non-null    float64
 6   subject              500 non-null    int64  
 7   city                 500 non-null    int64  
 8   marks_obtained       500 non-null    float64
dtypes: float64(4), int64(5)
memory usage: 39.1 KB
None
```

Final shape: (500, 9) with no missing values across all columns.

8. Final Preprocessed Dataset (Head)

```
df.head()
```

	student_id	age	class	section	study_hours_per_week	attendance_pct	subject	city	marks_obtained
0	1001	1.290644e-15	0	1	1.160013	1.483290e+00	1	1	1.476043
1	1002	1.484120e+00	2	2	1.179550	-1.044406e-15	4	2	-0.049622
2	1003	3.098371e-02	2	0	1.570302	-1.044406e-15	1	4	-0.985580
3	1004	1.484120e+00	3	2	-1.243109	1.358351e+00	2	0	-1.640209
4	1005	1.484120e+00	0	1	0.398047	-4.495902e-01	2	2	-0.336360

First 5 rows of the cleaned, encoded, and scaled dataset.

Result

Thus, the Student Result dataset was successfully preprocessed using Python by removing 12 duplicate records, imputing missing values (mean for age, study_hours_per_week, and attendance_pct; mode for city), label-encoding categorical features (class, section, subject, city), and standardizing continuous numerical attributes via z-score scaling. The resulting dataset (500 rows × 9 columns) is clean, consistent, and structurally validated for analytical modeling.